

Coverage-Constrained Relative Trajectory Planning for Autonomous Space Station Inspection in ROS 2

Rugang^a, OrbInspect Contributors^b

^a*OrbInspect Project, Shanghai, China*

^b*Open Research Software Collaboration, International*

Abstract

Autonomous inspection of large orbital structures requires planning algorithms that jointly account for relative orbital dynamics, sensor visibility, target coverage, collision avoidance, and fuel expenditure. This paper presents OrbInspect, a ROS 2-based simulation and planning framework for external inspection of space-station-scale structures, and formulates a coverage-constrained trajectory planning problem in the local-vertical/local-horizontal frame. The proposed planning workflow first samples inspection targets on a three-dimensional station model, constructs safe candidate viewpoints, solves an offline coverage-aware viewpoint ordering problem, and then generates a dynamically feasible trajectory under Clohessy–Wiltshire relative dynamics. The resulting trajectory is validated in ROS 2 with a Gazebo Harmonic visualization using a NASA International Space Station model. The current draft defines the formal problem, mission assumptions, constraints, and algorithmic evaluation protocol required before claiming full-coverage experimental results.

Keywords: spacecraft inspection, relative orbital dynamics, coverage planning, next-best-view planning, Clohessy–Wiltshire equations, ROS 2, Gazebo Harmonic

1. Introduction

Large orbital structures such as space stations, service platforms, and assembled space telescopes require periodic external inspection. Human extravehicular activity is expensive and hazardous, while fixed cameras provide limited coverage of complex external geometry. A small free-flying inspection

spacecraft can provide richer visual coverage, but planning its trajectory is a coupled guidance, navigation, control, perception, and safety problem.

The difficulty is not merely producing a visually appealing simulation. A research-grade inspection planner must answer the following questions: which surface elements must be inspected, from which viewpoints can they be observed, what trajectory connects these viewpoints under relative orbital dynamics, and how safety margins are maintained near large structures. In addition, the resulting experiment must produce reproducible trajectory, control, coverage, safety, and planning logs suitable for academic evaluation.

This paper draft positions OrbInspect around the following contribution: a coverage-constrained trajectory planner for large orbital structure inspection under Clohessy–Wiltshire relative dynamics, with validation in ROS 2 and Gazebo Harmonic. The present manuscript intentionally separates three layers:

1. formal problem definition and mission assumptions;
2. offline trajectory generation using a geometry-aware coverage model;
3. ROS 2/Gazebo replay and validation using the generated trajectory.

This separation avoids over-claiming from a short Gazebo visualization. Gazebo is used to validate visualization, logging, and execution interfaces; the coverage claim must come from the planner and its recorded metrics.

2. Related Work

Orbital inspection combines ideas from spacecraft relative motion, visual inspection planning, next-best-view planning, and safe control. Relative motion near a circular chief orbit is commonly modeled with the Clohessy–Wiltshire/Hill equations, which provide a tractable baseline for guidance design and trajectory optimization (Clohessy and Wiltshire, 1960). Higher-fidelity models can later be incorporated through numerical astrodynamics backends, but a linear relative-motion baseline is valuable for reproducible algorithm development.

Coverage planning and next-best-view methods are widely used in robotics for inspection of complex geometry (Scott et al., 2003). However, orbital inspection differs from terrestrial inspection because viewpoints are constrained by orbital dynamics, thruster authority, keep-out zones, line-of-sight blockage, and fuel. The planner must therefore optimize over both sensing utility and dynamic feasibility.

OrbInspect is designed to bridge this gap by combining a coverage model, a relative-dynamics trajectory generator, safety constraints, and ROS 2-based experiment logging. The NASA International Space Station mesh is used for visual realism, while simplified primitives are retained initially for occlusion and safety certificates until mesh-accelerated ray casting is implemented.

3. System Model and Mission Setting

3.1. Reference Frames

The inspection scenario is expressed in a local-vertical/local-horizontal (LVLH) frame attached to a nominal target orbit. The target structure is treated as fixed in this frame during one inspection pass. The chaser state is

$$x = [r_x \ r_y \ r_z \ v_x \ v_y \ v_z]^T, \quad (1)$$

where $r \in \mathbb{R}^3$ and $v \in \mathbb{R}^3$ are relative position and velocity in LVLH.

3.2. Relative Dynamics

For the baseline planner, chaser motion follows the Clohessy–Wiltshire equations with control acceleration $u = [a_x, a_y, a_z]^T$:

$$\dot{r}_x = v_x, \quad \dot{r}_y = v_y, \quad \dot{r}_z = v_z, \quad (2)$$

$$\dot{v}_x = 3n^2 r_x + 2nv_y + a_x, \quad \dot{v}_y = -2nv_x + a_y, \quad \dot{v}_z = -n^2 r_z + a_z, \quad (3)$$

where n is the chief mean motion. The discrete propagation map is denoted

$$x_{k+1} = f_{\text{CW}}(x_k, u_k, \Delta t). \quad (4)$$

This model is appropriate as a first-order relative-motion baseline for near-circular-orbit inspection studies and is consistent with standard mission engineering practice (Wertz et al., 2011).

3.3. Inspection Target Model

Let $\mathcal{T} = \{1, \dots, N\}$ be a finite set of surface inspection targets sampled from the station geometry. Each target i has a surface point p_i , normal n_i , and dwell-time accumulator τ_i . A target is inspected when it is visible for at least a required dwell time $\tau_{\min}$.

The current implementation can generate targets from simplified station primitives. The planned paper algorithm should extend this to mesh-aware target sampling on the real ISS model, with primitive geometry retained as a conservative safety envelope.

3.4. Mission Objective

The mission starts from a safe initial relative state x_0 and terminates when the required coverage threshold is achieved or when a maximum mission time is reached:

$$C_K = \frac{1}{N} \sum_{i=1}^N y_{i,K} \geq C_{\text{req}}, \quad (5)$$

where $y_{i,K} \in \{0, 1\}$ indicates whether target i has been inspected by the final time step K .

4. Problem Formulation

4.1. Visibility Conditions

A target i is visible from chaser pose x_k if all of the following hold:

$$\rho_{\min} \leq \|p_i - r_k\| \leq \rho_{\max}, \quad (6)$$

$$p_i \in \text{FOV}(r_k, q_k), \quad (7)$$

$$\arccos \left(\frac{(r_k - p_i)^\top n_i}{\|r_k - p_i\| \|n_i\|} \right) \leq \theta_{\max}, \quad (8)$$

$$\text{LOS}(r_k, p_i, \mathcal{O}) = 1, \quad (9)$$

where $\rho_{\min}$ and $\rho_{\max}$ define the camera range, q_k is the camera attitude or assumed station-facing boresight, $\theta_{\max}$ is the maximum allowable viewing angle, and $\mathcal{O}$ is the set of occluding station primitives or mesh triangles.

The dwell-time update is

$$\tau_{i,k+1} = \begin{cases} \tau_{i,k} + \Delta t, & \text{if target } i \text{ is visible at } k, \\ \tau_{i,k}, & \text{otherwise,} \end{cases} \quad (10)$$

and the inspection indicator is

$$y_{i,k} = \mathbf{1}\{\tau_{i,k} \geq \tau_{\min}\}. \quad (11)$$

4.2. Safety and Actuation Constraints

The planner must satisfy acceleration, speed, and keep-out constraints:

$$\|u_k\|_2 \leq u_{\max}, \quad (12)$$

$$\|v_k\|_2 \leq v_{\max}, \quad (13)$$

$$d_S(r_k) \geq d_{\text{safe}}, \quad (14)$$

where $d_S(r_k)$ is the signed distance from the chaser to the station safety envelope.

Additional mission constraints may include terminal conditions, sunlight constraints, communication geometry, plume impingement constraints, and camera slew-rate limits. These are outside the first implementation but should be represented in the final paper discussion.

4.3. Coverage-Constrained Trajectory Optimization

The full inspection planning problem can be written as

$$\min_{\{x_k, u_k, q_k\}_{k=0}^K} w_t K \Delta t + w_u \sum_{k=0}^{K-1} \|u_k\|_2 \Delta t + w_s \sum_{k=0}^K \phi(d_S(r_k)) + w_c(1 - C_K) \quad (15)$$

$$\text{s.t. } x_{k+1} = f_{\text{CW}}(x_k, u_k, \Delta t), \quad (16)$$

$$\|u_k\|_2 \leq u_{\max}, \quad (17)$$

$$\|v_k\|_2 \leq v_{\max}, \quad (18)$$

$$d_S(r_k) \geq d_{\text{safe}}, \quad (19)$$

$$y_{i,k} \text{ follows the visibility and dwell model}, \quad (20)$$

$$C_K \geq C_{\text{req}}, \quad (21)$$

where $\phi(\cdot)$ is a soft safety penalty used only when a relaxed version of the problem is solved. A hard-constrained implementation should enforce $d_S(r_k) \geq d_{\text{safe}}$ at all times.

This problem is mixed-integer and nonconvex because of visibility, occlusion, and coverage indicators. Therefore, the proposed first paper algorithm uses a structured offline decomposition.

5. Proposed Offline Planner

5.1. Planner Decomposition

The proposed planner decomposes the full problem into four stages:

1. target sampling on the station model;
2. safe candidate viewpoint generation;
3. coverage-aware viewpoint selection and ordering;
4. CW-feasible trajectory generation between selected viewpoints.

This decomposition is chosen because it produces interpretable paper metrics: coverage ratio, uncovered target set, planned Δv , executed Δv , minimum clearance, planning time, and mission duration.

5.2. Target Sampling

For mesh-based experiments, the station surface is sampled into target points p_i with normals n_i . The sampling density should be configurable by a target spacing parameter. For fair comparison, the same sampled target set is used by all methods in a scenario.

5.3. Candidate Viewpoint Generation

A candidate viewpoint z_j is generated on offset shells around the station:

$$z_j = p_i + \rho_j n_i, \quad (22)$$

where $\rho_j \in [\rho_{\min}, \rho_{\max}]$ and the candidate is accepted only if it satisfies keep-out constraints and does not lie inside the station safety envelope. Candidate viewpoints can also be generated from cylindrical or spherical shells surrounding major structural components.

For each accepted candidate z_j , a binary visibility vector $A_{ij} \in \{0, 1\}$ is computed:

$$A_{ij} = 1 \iff \text{target } i \text{ is visible from candidate } j. \quad (23)$$

5.4. Viewpoint Selection

The viewpoint subset can be selected by a weighted set-cover objective:

$$\min_{\alpha_j \in \{0,1\}} \sum_j c_j \alpha_j \quad (24)$$

$$\text{s.t.} \quad \sum_j A_{ij} \alpha_j \geq 1, \quad \forall i \in \mathcal{T}_{\text{req}}, \quad (25)$$

where c_j penalizes fuel estimate, poor safety margin, and view quality. If full set cover is too large, a greedy submodular approximation is used:

$$j^* = \arg \max_j \left[w_{\text{cov}} \Delta C_j - w_d d(r, z_j) - w_u \hat{\Delta} v(r, z_j) + w_m m_j \right], \quad (26)$$

where ΔC_j is new coverage, $d(r, z_j)$ is travel distance, $\hat{\Delta} v$ is an estimated transfer cost, and m_j is safety margin.

Algorithm 1 Offline coverage-constrained CW inspection planner

Require: Station mesh or primitive model $\mathcal{S}$, initial state x_0 , camera model, safety margin d_{safe} , coverage threshold C_{req}

Ensure: Planned trajectory $\mathcal{P}$ and inspection metrics

- 1: Sample surface targets $\mathcal{T} = \{(p_i, n_i)\}_{i=1}^N$
 - 2: Generate candidate viewpoints $\mathcal{V} = \{z_j\}_{j=1}^M$
 - 3: **for all** $z_j \in \mathcal{V}$ **do**
 - 4: Reject z_j if $d_{\mathcal{S}}(z_j) < d_{\text{safe}}$
 - 5: Compute visibility vector $A_{.j}$ using range, FOV, view angle, and line-of-sight constraints
 - 6: **end for**
 - 7: Select viewpoint subset $\mathcal{V}^*$ using weighted set cover or greedy submodular coverage
 - 8: Order $\mathcal{V}^*$ using CW-transfer edge costs
 - 9: Generate bounded-acceleration CW transfers between consecutive viewpoints
 - 10: Simulate planned trajectory and compute predicted coverage
 - 11: **if** $C_{\text{final}} < C_{\text{req}}$ **then**
 - 12: Add additional viewpoints for uncovered target clusters and replan
 - 13: **end if**
 - 14: Save trajectory, selected viewpoints, visibility matrix, and metrics
 - 15: **return** $\mathcal{P}$
-

5.5. Ordering and CW Transfer Generation

After selecting candidate viewpoints, their order is computed by solving a traveling-salesman-like problem with edge costs estimated from CW transfers. For each edge (i, j) , the planner computes a dynamically feasible transfer under bounded acceleration and rejects edges that violate safety margins.

5.6. Trajectory Output

The offline planner outputs a timestamped reference trajectory

$$\mathcal{P} = \{(t_k, r_k^{\text{ref}}, v_k^{\text{ref}}, u_k^{\text{ref}})\}_{k=0}^K. \quad (27)$$

This trajectory is saved before ROS execution and then replayed through the controller/dynamics stack. This is the proper order for the final paper: trajectory first, Gazebo validation second.

6. Simulation and Validation Protocol

6.1. Offline Planning Stage

The offline planner is run in Python before launching Gazebo. It must produce:

- sampled target set and normals;
- selected viewpoints;
- planned CW trajectory;
- predicted coverage over time;
- predicted Δv and clearance margins.

The planned outputs should be saved in the experiment directory `data/results/<timestamp>/`, including configuration snapshots and raw CSV logs.

6.2. ROS 2 Execution Stage

The planned trajectory is then supplied to the ROS 2 stack. The HCW dynamics node remains the source of truth for chaser state. The controller tracks the planned reference, the safety layer filters control commands, and the coverage node evaluates observed targets. Gazebo Harmonic visualizes the NASA ISS model and chaser model but does not provide spacecraft dynamics.

6.3. Evaluation Metrics

Each experiment should report:

$$C_{\text{final}} = \frac{N_{\text{inspected}}}{N}, \quad (28)$$

$$\Delta v_{\text{cum}} = \sum_k \|u_k\|_2 \Delta t, \quad (29)$$

$$d_{\text{min}} = \min_k d_S(r_k), \quad (30)$$

$$e_{\text{track,rms}} = \sqrt{\frac{1}{K} \sum_k \|r_k - r_k^{\text{ref}}\|_2^2}. \quad (31)$$

The final paper should compare at least four methods:

1. fixed waypoint baseline;
2. random safe viewpoint baseline;
3. greedy next-best-view baseline;
4. proposed coverage-constrained CW trajectory planner.

7. Expected Results and Figures

The current repository can generate trajectory, control, safety, coverage, and planner logs. However, the existing Gazebo videos should be treated as qualitative visualization until the offline planner reaches a verified coverage threshold on the target set.

The paper should include the following figures:

- sampled ISS inspection targets colored by inspected/uninspected state;
- planned three-dimensional trajectory in LVLH coordinates;
- executed trajectory compared with planned trajectory;
- coverage ratio versus time;
- cumulative Δv versus time;
- minimum clearance versus time;
- method comparison table over Monte Carlo runs.

The qualitative Gazebo figure should show the NASA ISS mesh and the chaser following the planned trajectory, but the quantitative claim should come from the logged coverage and safety metrics.

8. Limitations and Required Next Steps

The current software stack has several limitations that must be resolved before paper submission:

1. The current full-ISS video is not a proof of full coverage.
2. The real ISS mesh is used for visualization, while target generation and safety still rely primarily on simplified geometry.
3. The current offline trajectory mode is a waypoint demonstration, not a solved coverage-constrained optimization problem.

4. Gazebo is used for visualization only; it does not validate nonlinear orbital dynamics.
5. A higher-fidelity backend such as Basilisk should be used only as an optional validation case after the HCW baseline is reproducible.

The immediate technical priority is therefore to implement mesh-based target sampling, visibility matrix construction, viewpoint selection, CW-feasible transfer generation, and a result logger that proves the requested coverage threshold numerically before recording the final Gazebo video.

9. Conclusion

This draft formalizes the OrbInspect full-coverage inspection problem and defines the assumptions, constraints, objective, mission setting, and validation workflow needed for an academic paper. The central principle is that trajectory planning must precede visualization: a Python offline planner should generate a coverage-constrained, CW-feasible trajectory, and ROS 2/Gazebo should then validate execution, logging, and visual interpretation.

Future work will implement the mesh-aware planner and evaluate it against baseline methods using paper-grade logs, Monte Carlo trials, and repeatable Gazebo/RViz visualizations.

References

- Clohessey, W.H., Wiltshire, R.S., 1960. Terminal guidance system for satellite rendezvous. *Journal of the Aerospace Sciences* 27, 653–658.
- Scott, W.R., Roth, G., Rivest, J.F., 2003. View planning for automated three-dimensional object reconstruction and inspection. *ACM Computing Surveys* 35, 64–96.
- Wertz, J.R., Everett, D.F., Puschell, J.J., 2011. *Space Mission Engineering: The New SMAD*. Microcosm Press.